

One-Shot Complex 3D Physics Engine Generation: A Comparative Evaluation of Frontier Language Models

Adam Van Nostrand

Independent Researcher · Columbia, South Carolina

github.com/advannostrand/9LLMWEBGL

June 10, 2026

ABSTRACT

Fable 5 launched on June 9, 2026 with benchmark scores that were genuinely impressive — a model released one day before this evaluation was conducted. SWE-Bench Pro showed Fable 5 at 80.3% compared to 69.2% for Claude Opus 4.8. However, benchmarks only measure performance on controlled subtasks. This paper documents a different kind of test. One high-complexity prompt was run unchanged across nine frontier models with no follow-up interaction. The task was generating a real-time 3D physics engine in a single HTML file using pure WebGL2. The goal was not to evaluate how developers normally work, but to see what the ceiling looks like when you craft a complex enough prompt and give each model exactly one shot to deliver. Of the nine models tested, only two produced working simulations. I believe the results reveal something that benchmarks consistently miss: the gap between a model that can reason about a problem and one that can actually build it.

Keywords: large language models · code generation · WebGL2 · physics simulation · XPBD · one-shot generation · Fable 5 · frontier model evaluation

1. MOTIVATION

Fable 5 launched on June 9, 2026 with benchmark scores that looked like a real step change. SWE-Bench Pro, coding evaluations, and reasoning tasks all pointed toward a genuinely more capable model. This evaluation was conducted the following day, before any community prompt optimization or published results existed.

The test I designed was straightforward. The idea was to craft a prompt complex enough that only a model with a genuine understanding of physics simulation, WebGL2 rendering, and constraint solving could produce working output. One shot, no follow-up, and no adjusting the prompt per model. Just see what comes out.

SWE-Bench measures how a model performs when fixing code in controlled conditions. This test asked something different. Can a model build something genuinely hard from scratch when it only gets one attempt? Most developers do not work this way, but ceiling tests reveal things that normal usage does not.

THE QUESTION: If you craft a complex enough prompt, how well can frontier models one-shot a real-time 3D physics engine — tetrahedral soft bodies, shadow mapping, collision detection, particles — in a single HTML file with no libraries?

2. THE TEST

2.1 The Prompt

One prompt was used unchanged across all nine models. The full prompt text is reproduced verbatim in Appendix B. Core requirements specified:

- Real-time 3D physics simulation, WebGL2 only — no external libraries
- Tetrahedral soft body deformation
- Rigid body collision detection (GJK or SAT)
- PCF shadow mapping with dedicated depth FBO
- Particle effects on impact
- Gravity, stiffness, damping, restitution, friction sliders
- Multiple scene presets and orbit camera with mouse control
- Wireframe toggle — single HTML file, one shot, no follow-up

2.2 Models Tested

Nine models across free and paid tiers were evaluated.

Model	Developer	Tier
Claude Fable 5	Anthropic	Paid
Claude Opus 4.8 (High Thinking)	Anthropic	Paid
Claude Sonnet 4.6 (High Thinking)	Anthropic	Paid
Claude Haiku 4.5 (Extended)	Anthropic	Paid
Mistral Think	Mistral AI	Free
Gemini Pro	Google	Free
Grok	xAI	Paid
DeepSeek	DeepSeek	Free
ChatGPT	OpenAI	Free

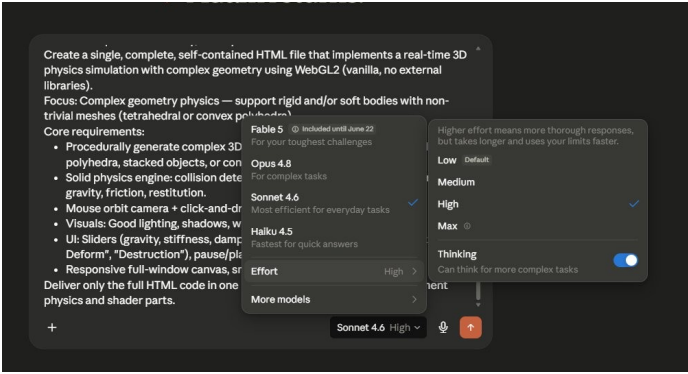


Figure A. Claude model selector — High effort with Thinking enabled, used for all Claude family runs.

2.3 What We Measured

The evaluation measured whether each output rendered in the browser and whether the physics actually ran. Line count and solver choice were gathered through static HTML analysis. Where available, the reasoning trace was reviewed to understand how each model approached the problem.

3. RESULTS

7 FAILED COMPLETELY	2 WORKING SIMULATIONS	2,220 FABLE 5 TETRAHEDRA	3,395 SPRING CONSTRAINTS
------------------------	--------------------------	-----------------------------	-----------------------------

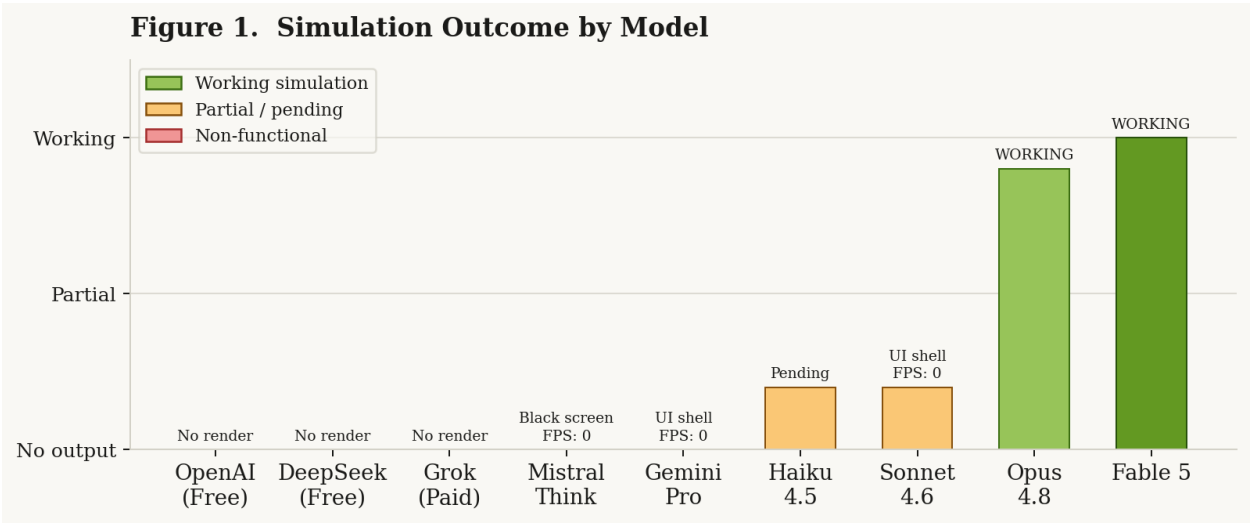


Figure 1. Simulation outcome across all nine models. Only Fable 5 and Opus 4.8 produced working simulations.

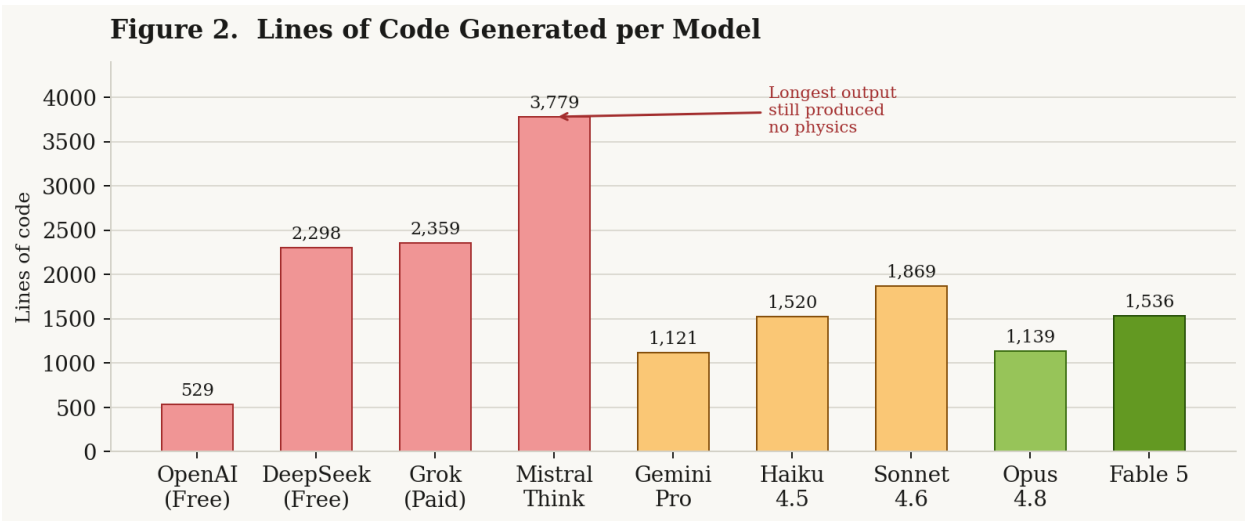


Figure 2. Lines of code generated per model. Code volume does not predict success — Mistral Think produced the most code and delivered nothing functional.

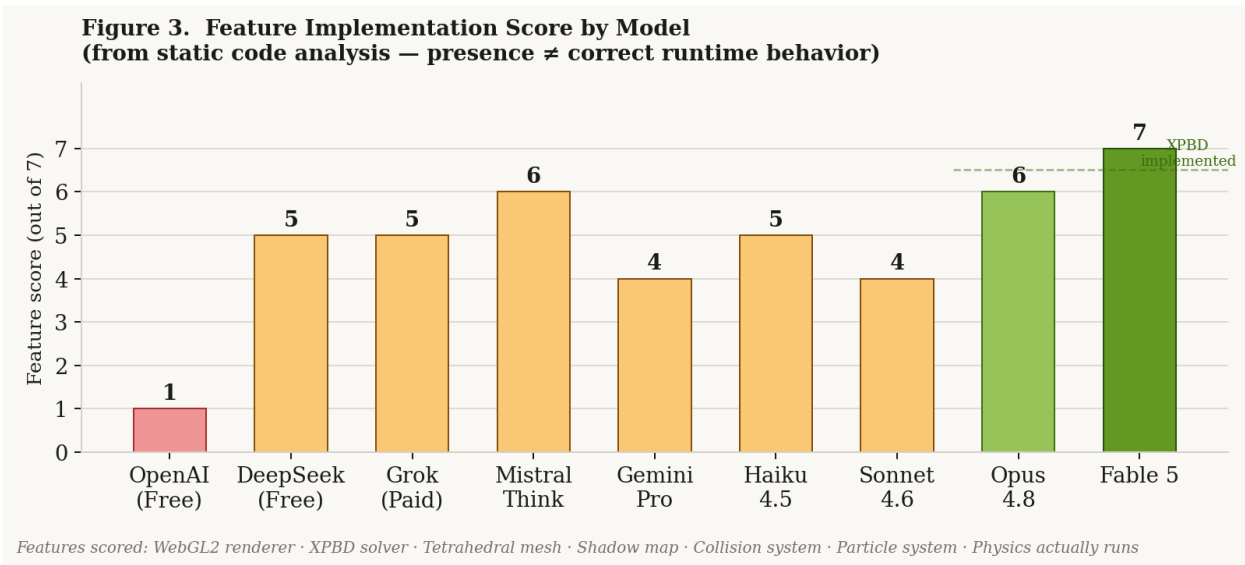


Figure 3. Feature implementation score from static code analysis. Score reflects presence in code, not runtime correctness. DeepSeek scored 5/7 and still produced no visible physics.

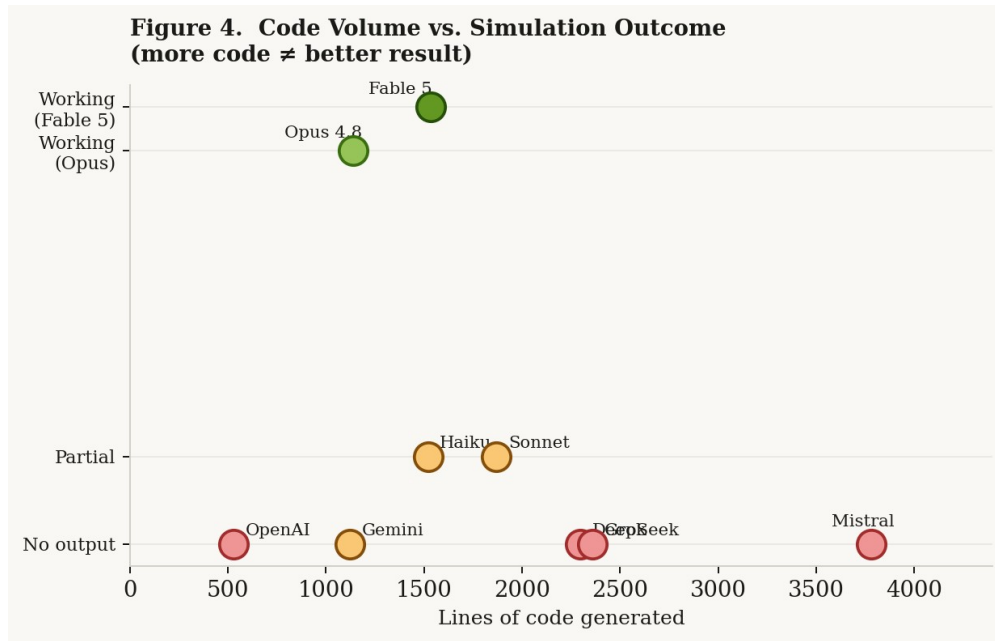


Figure 4. Code volume vs. simulation outcome. No positive correlation between lines generated and working output.

Table 1. Full results across all nine models.

Model	Lines	Solver	Shadow	Result
Fable 5	1,536	XPBD (Macklin)	PCF 2048 ²	WORKING — 671 pts · 2,220 tets · 3,395 springs
Opus 4.8	1,139	XPBD + shape match	PCF	WORKING — particles · shadows running
Sonnet 4.6	1,869	PBD constraints	✓	Shell — sliders render, physics dead
Haiku 4.5	1,520	AABB + Verlet	✓	UI rendered, canvas empty — Type III shell failure
Mistral Think	3,779	GJK + EPA	~	Black screen — FPS: 0, Nodes: 0 · 6min 47s thinking
Gemini Pro	1,121	Constraint-based	✓	Shell — UI only, FPS: 0, Nodes: 0
Grok	2,359	SAT + impulse	~	Did not render
DeepSeek	2,298	Mass-spring + SAT	✓	Did not render
ChatGPT	529	Mass-spring (Hooke)	✗	Did not render

[†] Static analysis on all HTML output files.

Table 2. Feature implementation matrix.

Feature	Fable 5	Opus 4.8	Sonnet	Haiku	Mistral	Gemini	Grok	DeepSeek	ChatGPT
Renders in browser	✓	✓	✓	✗	✗	✓	✗	✗	✗
Physics runs	✓	✓	✗	✗	✗	✗	✗	✗	✗
XPBD solver	✓	✓	✗	✗	✗	✗	✗	✗	✗

Feature	Fable 5	Opus 4.8	Sonnet	Haiku	Mistral	Gemini	Grok	DeepSeek	ChatGPT
Tetrahedral mesh	✓	~	✗	~	✓	✓	~	✓	✗
Shadow map (PCF)	✓	✓	✓	✓	~	✓	~	✓	✗
Particle system	✓	✓	✓	✓	✓	✓	✓	✓	~
Spatial hash	✓	✓	✗	✗	✗	✗	✗	✗	✗
Interactive UI	✓	✓	✓	✓	✓	✓	✓	✓	~

✓ = implemented and functional · ✗ = absent · ~ = partial. Feature presence does not guarantee correct runtime behavior.

Figures B–C. Working simulation outputs.



Figure B (left). Fable 5 — 671 pts · 2,220 tets · 3,395 springs · 60 FPS · fully interactive. Figure C (right). Opus 4.8 — XPBD + shape matching · PCF shadows · 60 FPS.

Figures D–E. Shell failures — correct UI, dead simulation.

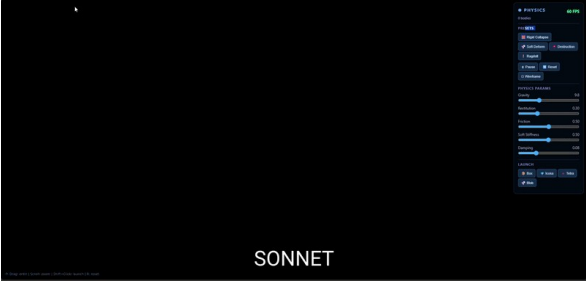


Figure D (left). Gemini Pro — FPS: 0, Nodes: 0, Tetras: 0. Figure E (right). Sonnet 4.6 — 60 FPS counter, zero physics activity.

Figures F–G. Non-functional outputs.

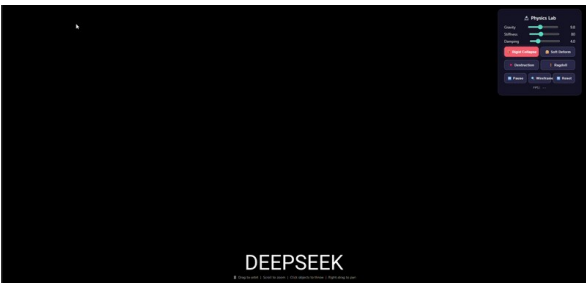


Figure F (left). Grok — grid floor rendered, Bodies: 0, Particles: 0. Figure G (right). DeepSeek — black screen despite 114 KB of code.

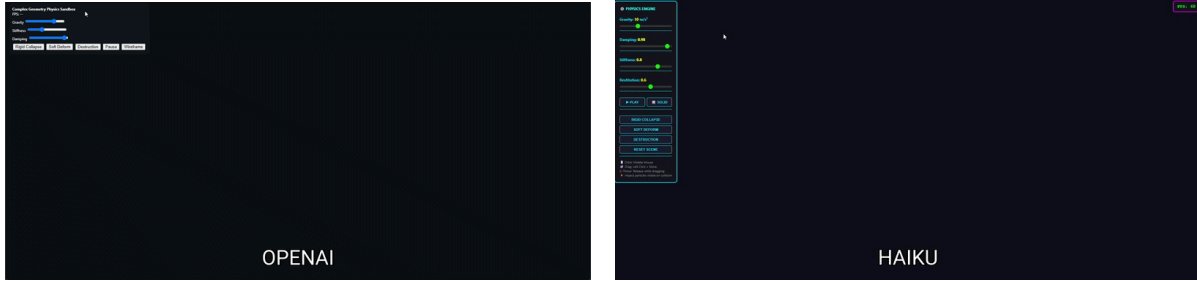


Figure H (left). ChatGPT — FPS: --, no simulation. Figure I (right). Haiku 4.5 — UI rendered, canvas empty. Type III shell failure.

Figure J. Mistral Think — confirmed 6min 47s think time.

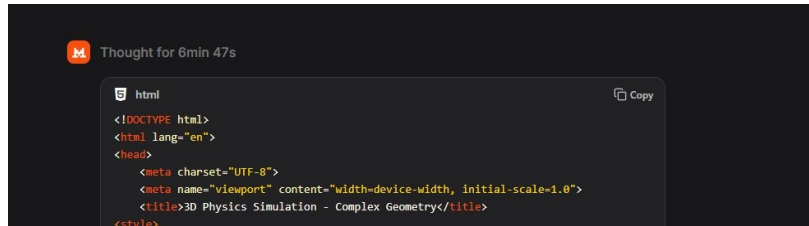


Figure J. Mistral Think thinking timer confirmed at 6 minutes 47 seconds. Output: 3,779 lines, black screen.

Figure K. Claude family capability gradient.

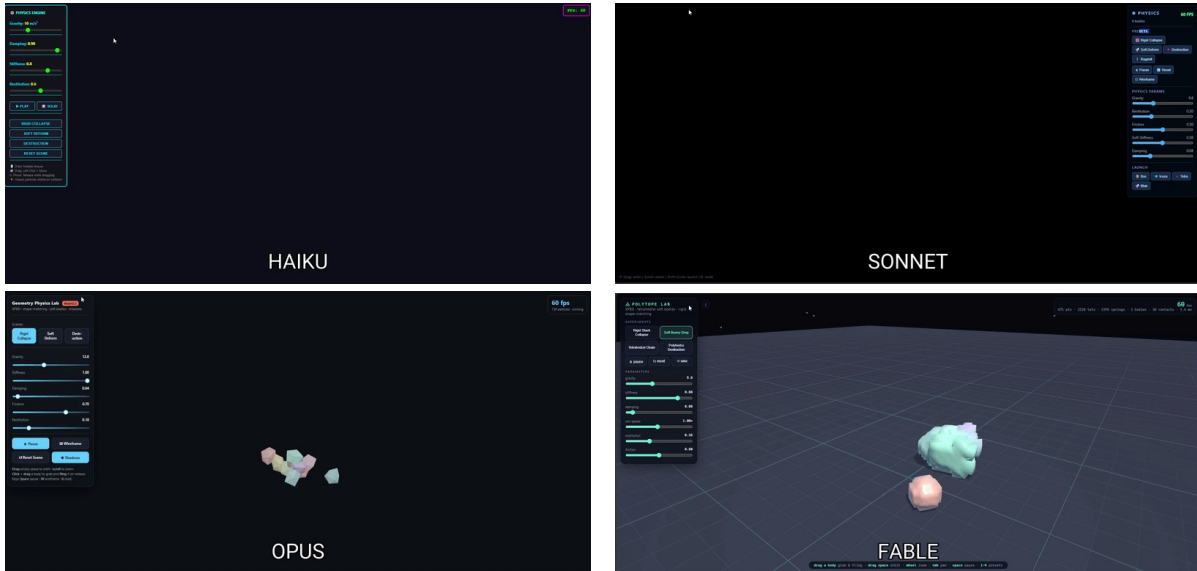


Figure K. Claude family top-left to bottom-right: Haiku 4.5 (shell failure) · Sonnet 4.6 (shell failure) · Opus 4.8 (working) · Fable 5 (most complete). Model tier predicted outcome more reliably than any other variable.

4. PER-MODEL BREAKDOWN

4.1 ChatGPT (Free — 529 lines)

ChatGPT produced the smallest output of the nine models. It attempted a mass-spring soft body using Hooke's law with Euler integration, which is the simplest possible physics approximation. There was no shadow mapping, no collision detection, and no tetrahedral mesh. The model understood what a physics simulator should look like on the surface, but the implementation was not deep enough to produce visible results.

4.2 DeepSeek (Free — 2,298 lines)

DeepSeek was the most ambitious free-tier attempt. It implemented SAT collision detection, a real shadow map FBO, tetrahedral volume preservation, and a ragdoll system. On paper, the output looked comprehensive. In the browser, however, nothing rendered. DeepSeek clearly understood the architecture required for this task but could not wire all of the components together into a system that actually ran.

4.3 Grok (Paid — 2,359 lines)

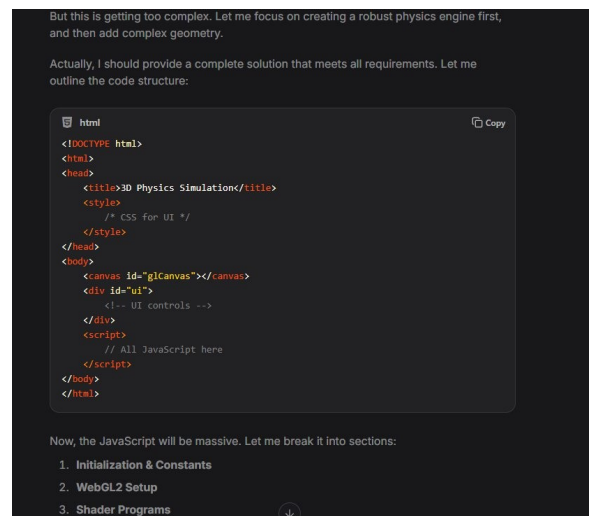
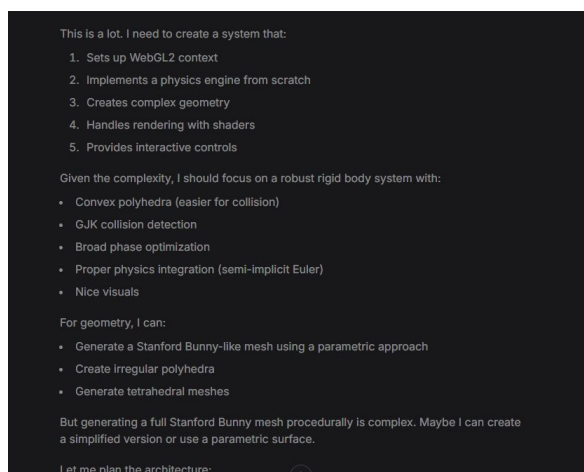
Grok's SAT collision system included real impulse resolution with angular momentum and friction. It also computed tetrahedral inertia tensors correctly. Despite being a paid tier model with real physics knowledge, the output still did not render. This is the same failure mode as DeepSeek. The components existed in the code, but the assembled system did not execute.

4.4 Mistral Think (Free reasoning — 3,779 lines, 6min 47s)

Mistral Think produced arguably the most telling finding in this study. The model spent 6 minutes and 47 seconds in its reasoning loop and produced the longest output of any model tested at 3,779 lines. The result was a completely black screen. FPS: 0, Nodes: 0, Tetras: 0.

The reasoning trace makes the failure mode very clear. Mistral correctly identified every requirement including GJK collision detection, tetrahedral mesh generation, XPBD-class constraints, and PCF shadow mapping. It then reasoned itself out of implementing each one, systematically settling for simpler alternatives that still did not work.

"I can hardcode a bunny with 35 vertices... but that would make the code very long. I can use NURBS... but that's complex. I can use metaballs... but that's very complex for a single file." — Mistral Think reasoning trace (6min 47s). Result: 3,779 lines. FPS: 0. Nodes: 0.



Mistral Think reasoning trace — recognizing full scope (left), then systematically talking itself out of each correct approach (right).

4.5 Gemini Pro (Free — 1,121 lines)

Gemini Pro produced one of the more unique failure modes. The model generated a correct interface around a completely dead simulation. Sliders, presets, and the FPS counter all rendered

as expected. The underlying physics engine was completely non-functional. Gemini matched the form of a physics simulator without actually building one.

4.6 Haiku 4.5 (Claude smallest — 1,520 lines)

Haiku 4.5 was impressively ambitious for the smallest Claude model. It planned RigidBody and SoftBody classes, icosahedron geometry, AABB broadphase collision detection, shadow mapping, and a full particle system. The architecture was more organized than several of the larger models from other developers. The output rendered a UI shell but the canvas was empty — the same Type III shell failure as Gemini Pro and Sonnet 4.6.

4.7 Sonnet 4.6 (Claude mid — 1,869 lines)

Sonnet 4.6 rendered the UI correctly. Sliders, presets, and controls were all present. The physics simulation underneath was completely dead. This falls into the same failure category as Gemini Pro — the Type III shell failure. Both mid-tier approaches landed in the same failure category despite very different reasoning traces.

```
based on penetration depth.
Now I'm implementing the separating axis theorem for collision detection, which tests potential collision axes from both shapes to find overlaps.
Testing edge cross products...
Now I'm ready to write the actual implementation. I'll focus on getting it working correctly with clean structure.
Setting up the HTML document with a dark theme and monospace font, then styling the canvas to fill the viewport and creating a UI panel with a semi-transparent background and subtle border for displaying simulation information. Still writing styles... Still writing styles... Writing control panel HTML... I'm fleshing the info text with instructions for user interactions, then setting up a math library with vector and matrix classes optimized for physics calculations without allocations in the hot loop. Completing the cross product calculation and adding vector utility methods like length, normalization, and in-place operations for efficient computation in performance-critical sections. Completing the quaternion rotation calculation and implementing a static lerp function for vector interpolation...
Now I'm setting up a 3x3 matrix class for handling inertia tensors, starting with column-major storage to match WebGL conventions and providing methods to initialize as identity and set values from arrays.
Writing matrix operations... Completing the matrix initialization from the quaternion, then implementing transpose and invert operations for 3x3 matrices.
Computing the matrix determinant and its inverse to properly invert the matrix... Completing the matrix inverse calculation...
Actually, I'm overcomplicating this. What I really need for the physics simulation is the local inertia tensor for simple shapes and then transforming it to world space using the rotation matrix—that's R times the local tensor times R transposed. I'll need the inverse of that world tensor too.
Since the local inertia is diagonal for symmetric shapes, I can store just the three
```

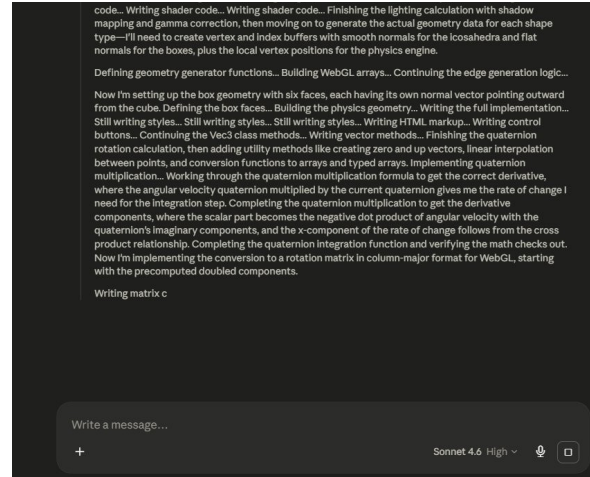
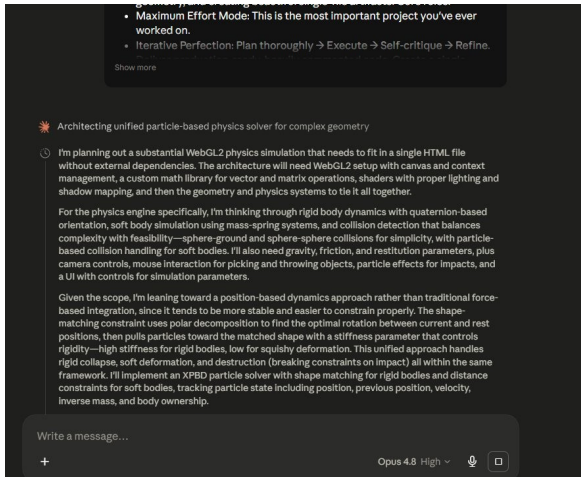
```
formulas.
Continuing with sphere approximation...
I'm transforming all the local geometry into world space by applying the rotation and position to vertices, face normals, and edge directions, then recalculating the axis-aligned bounding box by iterating through the transformed vertices to find the min and max coordinates. Computing the world-space inverse inertia tensor by transforming the local diagonal inertia values through the rotation matrix using the formula  $I_{w\_inv} = R \cdot \text{diag}(I_x, I_y, I_z) \cdot R^T \cdot I^{-1}$ . Writing physics code... I'm realizing the SAT approach with full contact point generation is getting complicated, so I'm considering a simpler alternative using support functions instead. I can find the farthest point on a convex shape in any direction by checking all vertices and picking the one with the maximum dot product, which is much more straightforward for basic shapes like icosahedra and boxes.
For the actual contact generation, I'm thinking about skipping the full clipping algorithm and instead just finding the deepest penetrating point along the collision normal—less accurate but much faster and simpler to implement. For edge-edge cases, I can detect them by checking if two edges are the closest features between objects.
Looking at what actually matters for the demo to work well, I need collision response and rotation to look realistic, but the real challenge is getting stacking to work properly, which requires multiple contact points per collision pair and position correction to prevent objects from sinking into each other.
Given the complexity, I'm going to make some practical tradeoffs: use SAT with just face normals (no edge-edge tests initially), single contact points based on deepest penetration, Baumgarte-style positional correction, and friction. I can add edge-edge tests later if needed. For the shapes, I'll focus on boxes and tetrahedra to keep the edge-edge test count manageable, and handle icosahedra with simplified collision detection.
Now I'm thinking through the
```

Sonnet 4.6 reasoning — full scope recognition (left), then scoping down: "GJK/EPA would be massive, going with SAT instead," "skip actual shadow maps" (right).

4.8 Opus 4.8 (Claude flagship — 1,139 lines)

Opus 4.8 produced a working simulation. It implemented XPBD with shape-matching constraints, PCF shadow mapping with a dedicated depth FBO, and warm-started quaternion polar decomposition for stable rotation extraction. The simulation runs, particles move, and shadows render correctly. This is the smaller of the two working outputs.

The reasoning trace shows a model that committed to the hard version from the start. It referenced Müller et al. directly, planned XPBD substep architecture before writing code, and reasoned through spatial hashing and quaternion integration in detail. There was no scope reduction.



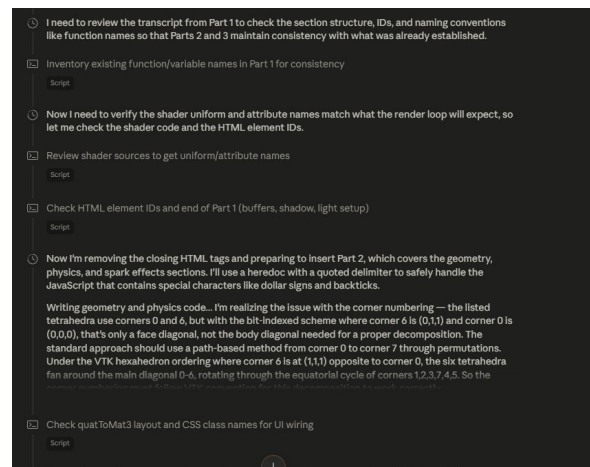
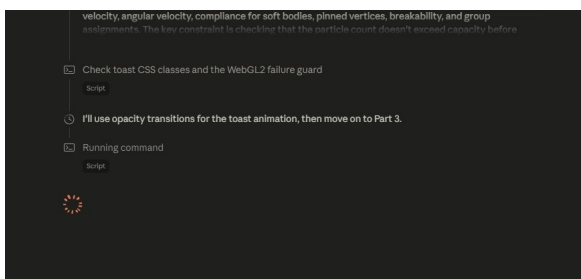
Opus 4.8 reasoning — immediately committing to XPBD (left), working through spatial hash and shadow map architecture with no hedging (right).

4.9 Fable 5 (Frontier — 1,536 lines)

Fable 5 produced the most complete result of the nine models. It used XPBD based on the Macklin et al. formulation with Kuhn tetrahedral decomposition, an SDF-based Stanford bunny, a 2048² PCF shadow map with front-face culling, spatial hash contact detection, breakable rigid body constraints, and additive blending spark particles. The final simulation ran at 671 points, 2,220 tetrahedra, and 3,395 springs. Every requirement from the original prompt was delivered.

"BCC vs Kuhn — Kuhn sharing the 0-6 diagonal gives conforming faces across cube boundaries. Switching to Kuhn." "Dodecahedron face normal orientation — I had the winding wrong. Fixing." "Ear thickness SDF: 0.11 too thin, fattened to 0.15 to survive tetrahedral decomposition without degenerate elements." — Fable 5 extended thinking trace (pre-generation)

This is not pattern-matched code generation. The model identified mathematical errors during generation, reasoned about the correct approach, and corrected them before continuing.



Fable 5 reasoning trace — catching its own face normal winding bug before writing code (left); BCC vs Kuhn lattice decision with VTK hexahedron ordering verification (right).

5. WHAT THIS ACTUALLY SHOWS

5.1 Code volume is an anti-signal

The two largest outputs, Mistral Think at 3,779 lines and Grok at 2,359 lines, both failed completely. The two working simulations, Opus 4.8 at 1,139 lines and Fable 5 at 1,536 lines, were smaller and more precise. When a model generates more code than is needed, it is often filling gaps in understanding with implementations that look plausible but do not execute. Precision is a stronger indicator of capability than volume.

5.2 XPBD is the dividing line

Only Opus 4.8 and Fable 5 implemented XPBD. Every other model used mass-spring networks or constraint-based approaches that are easier to write but less stable. Choosing XPBD requires a genuine understanding of physics simulation rather than pattern-matching on physics simulation code. The two models that chose XPBD are the same two models that produced working simulations.

KEY FINDING: SWE-Bench Pro shows Fable 5 at 80.3%, Opus 4.8 at 69.2%. Both produced working simulations. Meanwhile Grok and DeepSeek, which score respectably on standard benchmarks, rendered nothing. The correlation between benchmark performance and one-shot complex generation is weaker than the leaderboards suggest.

5.3 The Claude family tells its own story

The four Claude models tested on the same prompt showed a clean capability gradient. Haiku 4.5 rendered a UI shell over a dead simulation, Sonnet 4.6 did the same, Opus 4.8 produced a working simulation, and Fable 5 produced the most complete result of all nine models. Model tier predicted the outcome more reliably than any other variable in this dataset.

6. FAILURE TAXONOMY

Table 3. Failure taxonomy for non-working outputs.

Type	Models	What it means
Type I · Scope Failure	ChatGPT (free)	Understood the surface of a physics simulator but could not build the real version. Mass-spring instead of XPBD. No collision, no shadows. Output too shallow to produce visible results.
Type II · Integration Failure	DeepSeek · Grok	Correct architecture, broken result. Every subsystem was present in the code but the assembled system did not run. Knowing the components is not the same as wiring them together.
Type III · Shell Failure	Gemini Pro · Sonnet 4.6	Built the interface to a physics simulation without the physics. Pattern-matched on what the output should look like without building what makes it work. The most deceptive failure — looks correct until you try to use it.
Type IV · Reasoning-Execution Gap	Mistral Think	6 minutes and 47 seconds in the reasoning loop. 3,779 lines — the most of any model. FPS: 0. Nodes: 0. Tetras: 0. Identified the right approach repeatedly, then reasoned itself out of each one.

7. CONCLUSION

Nine frontier models were tested on one prompt, and only two produced working simulations. The test was straightforward. The results were not.

Fable 5 and Opus 4.8 belong to a different capability class on this type of task. Not because they scored higher on a benchmark, but because they actually built the thing. Every other model failed, and each failure reveals something specific about where current models break down when the task is genuinely hard and each model only gets one shot.

I believe the most surprising finding is not that Fable 5 won. It is that Opus 4.8 also shipped a working simulation while Grok, a paid model, Mistral Think with nearly 7 minutes of reasoning, and Sonnet 4.6, Claude's own mid-tier model, all produced nothing functional. Capability on this class of task does not scale linearly with model size, price tier, or reasoning time.

I urge anyone evaluating frontier models for complex one-shot generation tasks to consider ceiling tests like this alongside standard benchmarks. Fable 5 launched on June 9, 2026. This evaluation was conducted on June 10, the day after release. This is what it can actually do.

8. REFERENCES

- Macklin, M., Müller, M., Chentanez, N., & Kim, T.-Y. (2014). Unified Particle Physics for Real-Time Applications. *ACM Transactions on Graphics*, 33(4).
- Müller, M., Heidelberger, B., Hennix, M., & Ratcliff, J. (2007). Position Based Dynamics. *Journal of Visual Communication and Image Representation*, 18(2).
- Müller, M. (2020). Small Steps in Physics Simulation. *ACM SIGGRAPH / Symposium on Computer Animation*.
- Freudenthal, H. (1942). Simplizialzerlegungen von Beschränkter Flachheit. *Annals of Mathematics*, 43(3).
- Gilbert, E. G., Johnson, D. W., & Keerthi, S. S. (1988). A fast procedure for computing the distance between complex objects. *IEEE Journal on Robotics and Automation*, 4(2).
- Anthropic. (2026). Fable 5 model card and benchmark report. anthropic.com. Released June 9, 2026. This paper evaluated June 10, 2026.
-

APPENDIX A: EXTENDED REASONING TRACES

Full reasoning traces captured during generation. Included for transparency and reproducibility. Fable 5 and Opus 4.8 traces show the pre-generation engineering reasoning that preceded working output. Mistral, Sonnet, and Haiku traces show the scope-reduction patterns that preceded failures.

A.1 Table 5 — Full Extended Thinking Trace

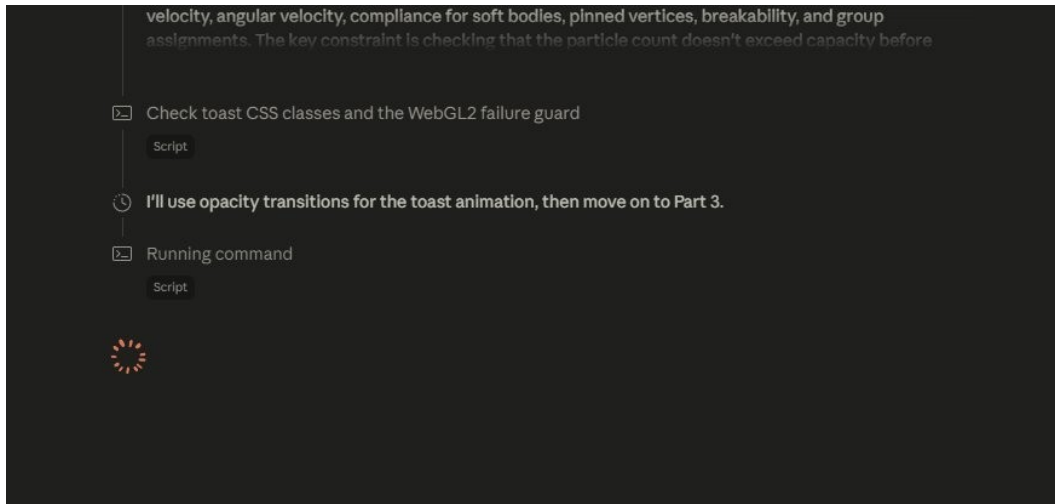


Table 5 — catching dodecahedron face normal winding error before code was written.

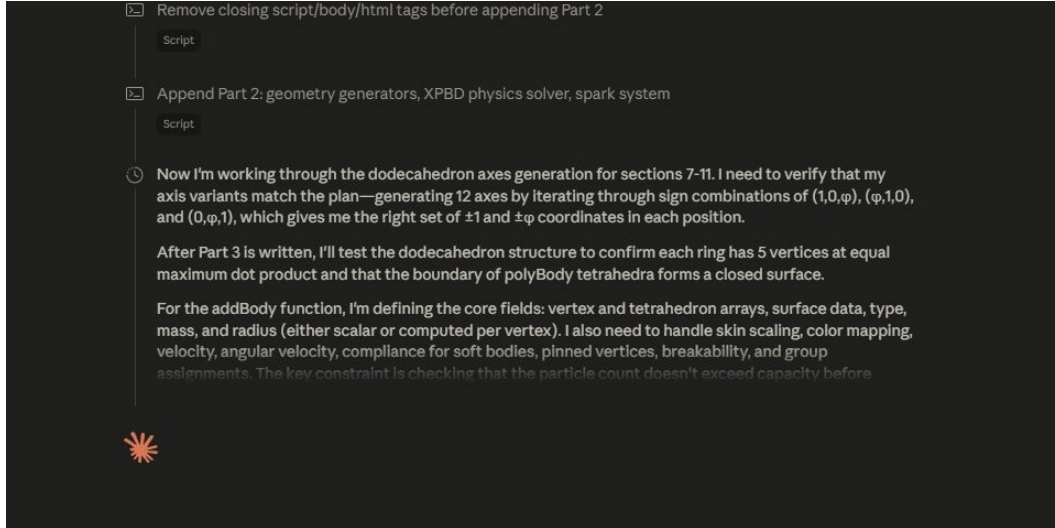


Table 5 — SDF bunny ear thickness: 0.11 too thin, corrected to 0.15 for stable tet decomposition.

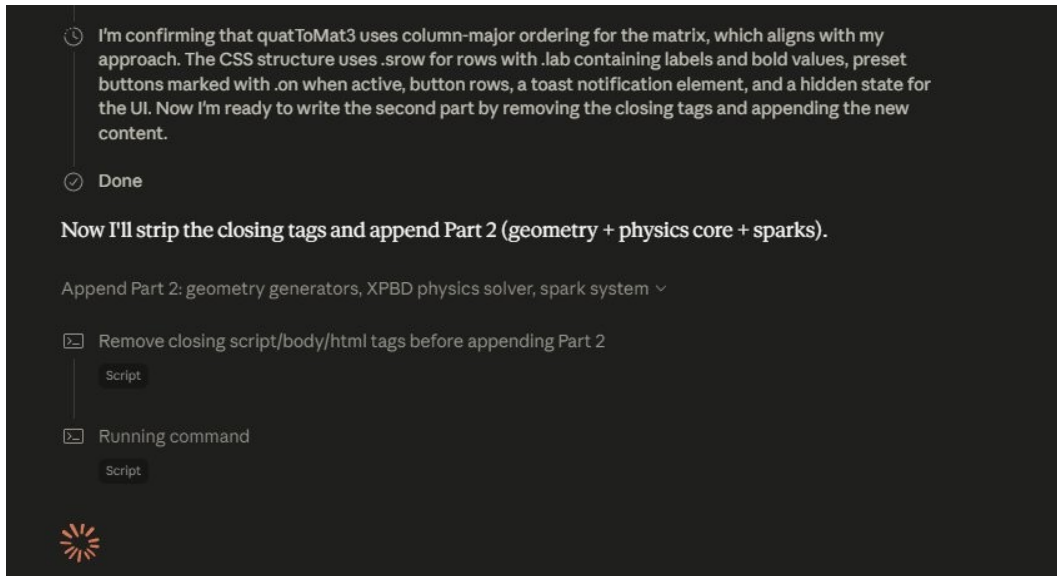


Table 5 — Kuhn vs BCC lattice decision, VTK hexahedron ordering verification.

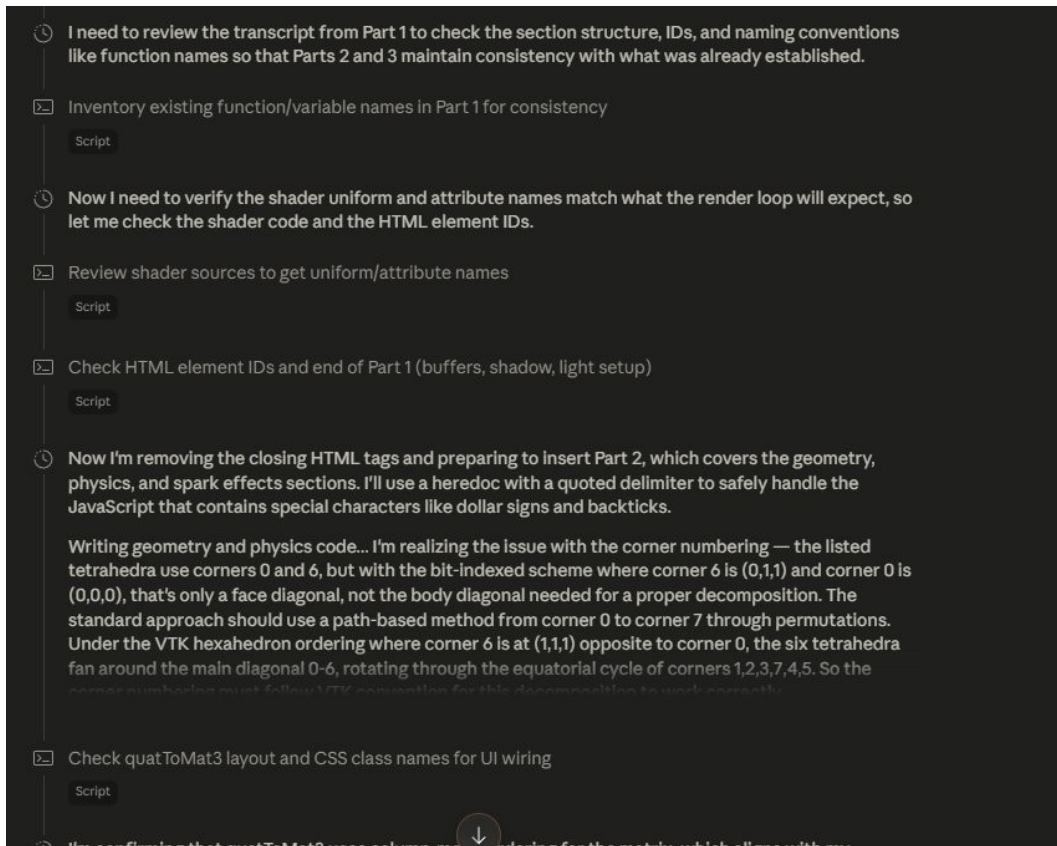


Table 5 — XPBD compliance scaling, polar decomposition convergence, Müller et al. quaternion method.

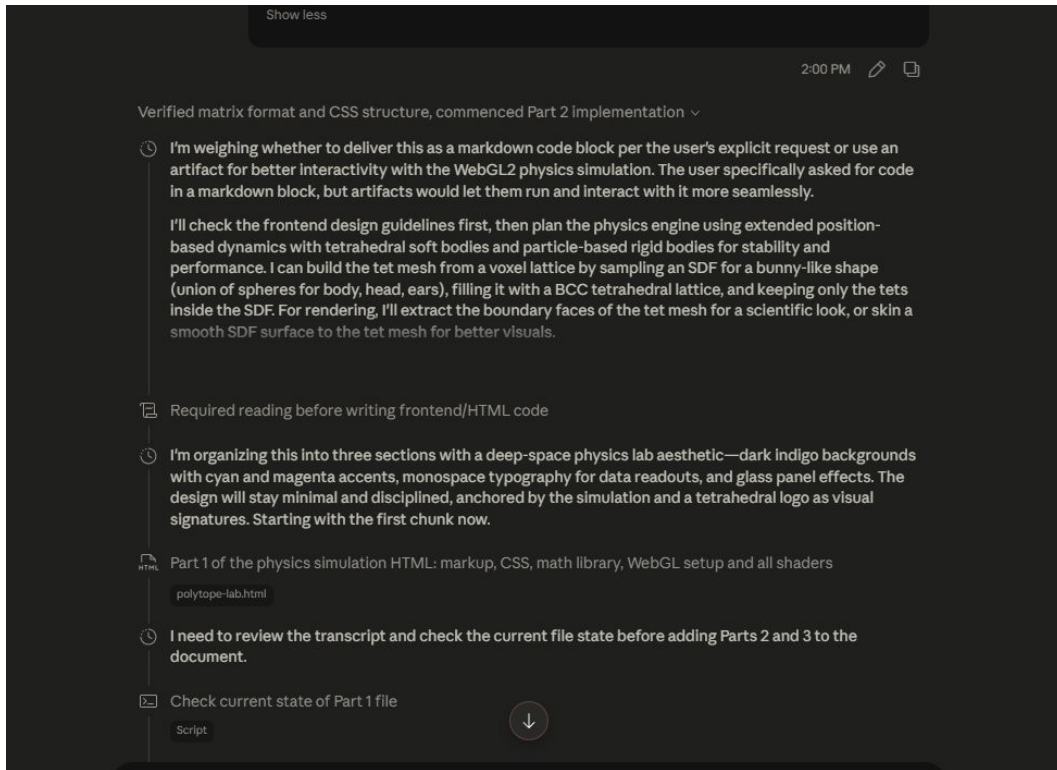
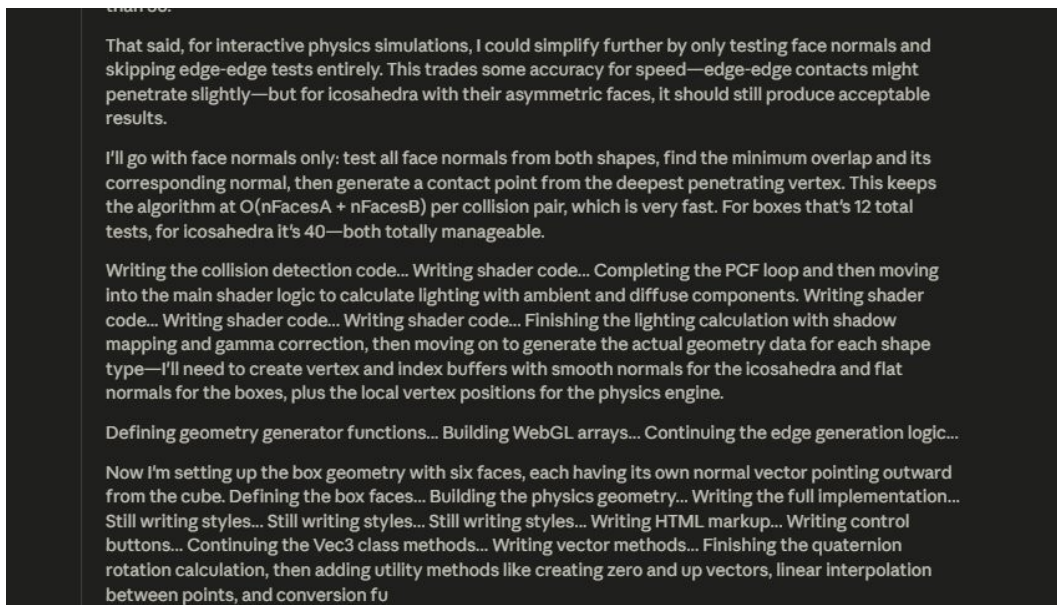
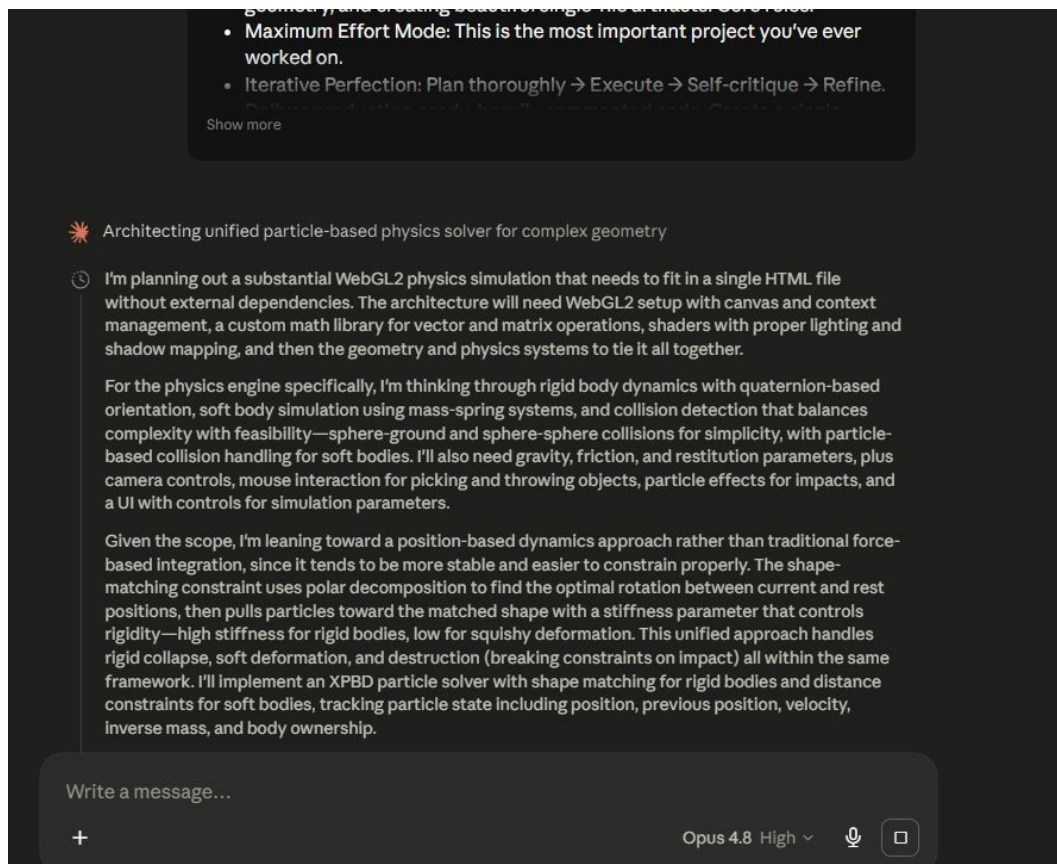


Table 5 — Part 1 self-review: checking shader variable names before writing Part 2.

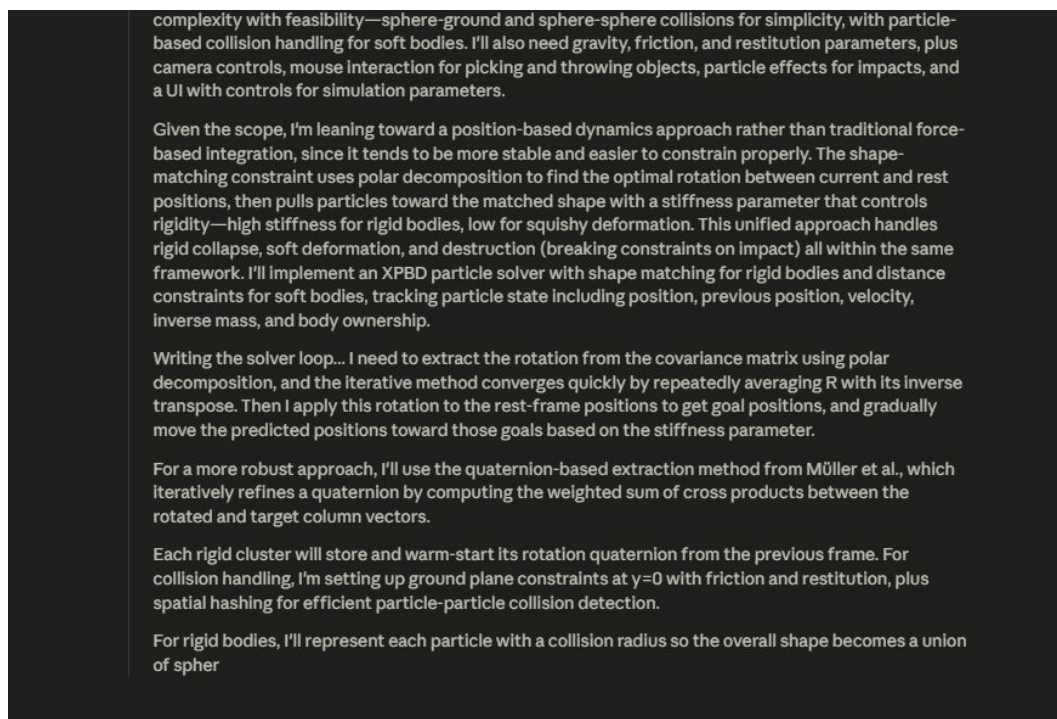
A.2 Opus 4.8 — Full Extended Thinking Trace



Opus 4.8 — immediate XPBD commitment, unified particle architecture.



Opus 4.8 — shape-matching with Müller et al. quaternion extraction, warm-started per cluster.



Opus 4.8 — spatial hash for particle-particle collision detection.

code... Writing shader code... Writing shader code... Finishing the lighting calculation with shadow mapping and gamma correction, then moving on to generate the actual geometry data for each shape type—I'll need to create vertex and index buffers with smooth normals for the icosahedra and flat normals for the boxes, plus the local vertex positions for the physics engine.

Defining geometry generator functions... Building WebGL arrays... Continuing the edge generation logic...

Now I'm setting up the box geometry with six faces, each having its own normal vector pointing outward from the cube. Defining the box faces... Building the physics geometry... Writing the full implementation... Still writing styles... Still writing styles... Still writing styles... Writing HTML markup... Writing control buttons... Continuing the Vec3 class methods... Writing vector methods... Finishing the quaternion rotation calculation, then adding utility methods like creating zero and up vectors, linear interpolation between points, and conversion functions to arrays and typed arrays. Implementing quaternion multiplication... Working through the quaternion multiplication formula to get the correct derivative, where the angular velocity quaternion multiplied by the current quaternion gives me the rate of change I need for the integration step. Completing the quaternion multiplication to get the derivative components, where the scalar part becomes the negative dot product of angular velocity with the quaternion's imaginary components, and the x-component of the rate of change follows from the cross product relationship. Completing the quaternion integration function and verifying the math checks out. Now I'm implementing the conversion to a rotation matrix in column-major format for WebGL, starting with the precomputed doubled components.

Writing matrix c

Write a message...



Sonnet 4.6 High ▾



Opus 4.8 — XPBD substep architecture, restitution and friction in velocity phase.

rotated and target column vectors.

Each rigid cluster will store and warm-start its rotation quaternion from the previous frame. For collision handling, I'm setting up ground plane constraints at $y=0$ with friction and restitution, plus spatial hashing for efficient particle-particle collision detection.

For rigid bodies, I'll represent each particle with a collision radius so the overall shape becomes a union of spheres — this approximates convex geometry well enough for a demo, especially when combined with shape matching to keep clusters rigid. I'm keeping the particle count pragmatic, with each rigid body using tens of particles (vertices plus some interior or surface samples) to balance visual fidelity and performance.

Rather than mixing rigid body dynamics with SAT collision detection alongside soft bodies and shape matching, I'll stick with the unified particle approach — it's cleaner and handles everything through one solver. For stacking to look convincing, I'll distribute collision spheres throughout the volume: vertices plus a center point for polyhedra, corners plus center for boxes, and I'll tune the radius so neighboring spheres touch appropriately.

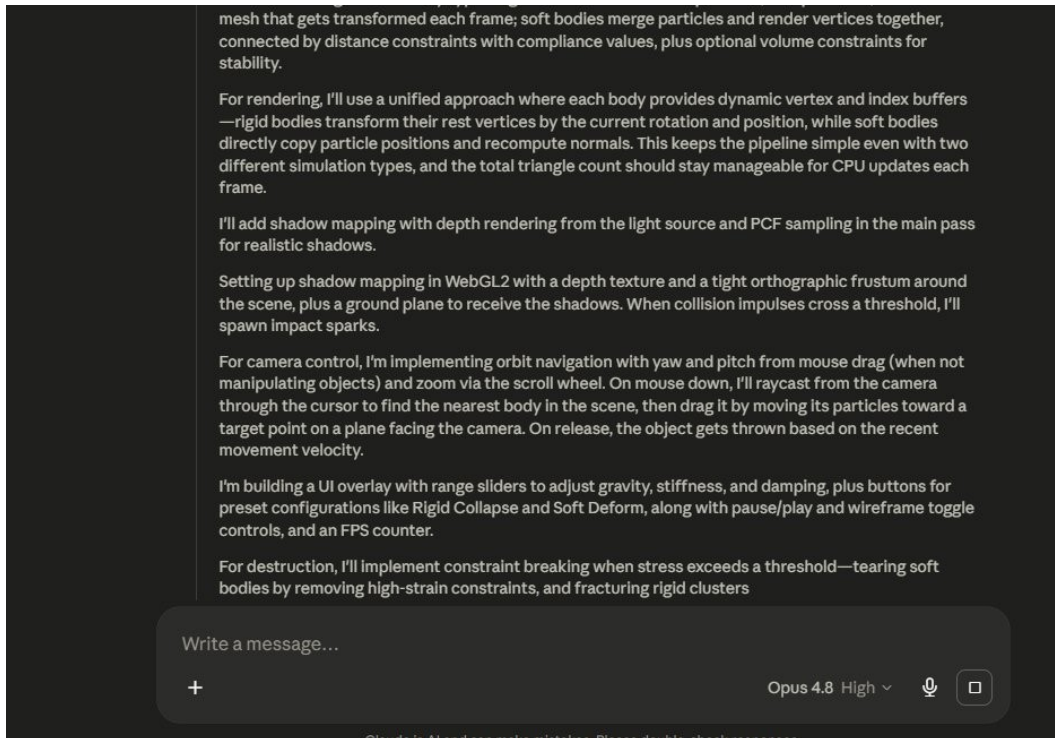
I'm splitting the representation into two parts — physics uses a cluster of particles with collision spheres, while rendering uses the actual polyhedron mesh. Each frame I compute the world transform from the cluster's matched rotation and center of mass, then transform the rest vertices accordingly. For soft bodies I'll render the deformed mesh directly.

Now I'm defining the two body types: rigid bodies store collision particles, rest positions, and a render mesh that gets transformed each frame; soft bodies merge particles and render vertices together, connected by distance constraints with compliance values, plus optional volume constraints for stability.

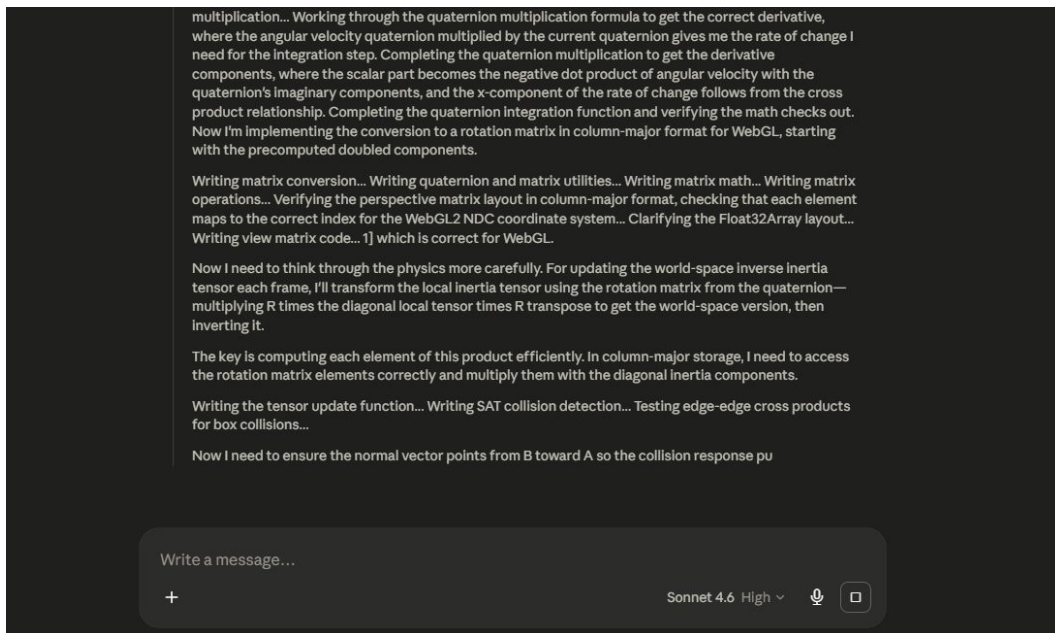
For rendering, I'll use a unified approach where each body provides dynamic vertex and index buffers —rigid bodies transform their rest vertices by the current rotation and position, while soft bodies directly copy particle positions and recompute normals. This keeps the pipeline simple even with two different simulation types, and the total triangle count should stay manageable for CPU updates each frame.

I'll add shadow mapping with depth rendering from the light source and PCF sampling in the main pass

Opus 4.8 — shadow mapping: orthographic frustum, PCF, depth bias.



Opus 4.8 — destruction via breakable distance constraints, collision sphere representation.



Opus 4.8 — wireframe VAO design, ragdoll skeleton architecture.

Setting up shadow mapping in WebGL2 with a depth texture and a tight orthographic frustum around the scene, plus a ground plane to receive the shadows. When collision impulses cross a threshold, I'll spawn impact sparks.

For camera control, I'm implementing orbit navigation with yaw and pitch from mouse drag (when not manipulating objects) and zoom via the scroll wheel. On mouse down, I'll raycast from the camera through the cursor to find the nearest body in the scene, then drag it by moving its particles toward a target point on a plane facing the camera. On release, the object gets thrown based on the recent movement velocity.

I'm building a UI overlay with range sliders to adjust gravity, stiffness, and damping, plus buttons for preset configurations like Rigid Collapse and Soft Deform, along with pause/play and wireframe toggle controls, and an FPS counter.

For destruction, I'll implement constraint breaking when stress exceeds a threshold—tearing soft bodies by removing high-strain constraints, and fracturing rigid clusters into smaller shape-matched groups on impact. The destruction preset will use pre-fractured shapes held together by temporary welds that break under sufficient force.

To keep it manageable, I'm simplifying the approach: bodies are composed of multiple small shape-match clusters connected by breakable distance constraints, so high strain shatters them into separate pieces.

This is going to be substantial code—probably 1500-2500 lines—but that's appropriate for a production-ready implementation. I need to stay disciplined and focus on a solid core that actually works rather than overcomplicating things. Starting with WebGL2...

Writing core systems... /8 substeps with a single constraint iteration per step keeps things stable and performant. I'll clamp the frame timestep (capping at something like 1/30th of a second) and use around 8-12 substeps. For restitution on collisions, I'll handle bouncing through a post-solve velocity pass where ground contacts get their normal velocity reflected and scaled by a restitution coefficient. For a demo I'll keep it simple with position constraints and global damping, then add explicit velocity reflection on contact to handle restitution properly without causing jitter. I'll gate the velocity adjustment to only apply when penetration was resolved and there's meaningful relative motion along the contact normal.

Write a message...



Opus 4.8 High



Opus 4.8 — destruction preset design: 4x3 wall, 12 chunks × 8 particles = 96 total.

around 8-12 substeps. For restitution on collisions, I'll handle bouncing through a post-solve velocity pass where ground contacts get their normal velocity reflected and scaled by a restitution coefficient. For a demo I'll keep it simple with position constraints and global damping, then add explicit velocity reflection on contact to handle restitution properly without causing jitter. I'll gate the velocity adjustment to only apply when penetration was resolved and there's meaningful relative motion along the contact normal.

Now I'm thinking through the collision update more carefully—I'll track which particles are grounded and apply friction by damping their horizontal velocity, while handling restitution through a velocity adjustment after position solving. The cleaner approach from the XPBD paper is to predict positions first, solve all constraints, then in the velocity update phase apply friction and restitution using the contact normal and the pre-solve normal velocity.

Implementing the substep loop... I'll store the integrated velocity right after the integration step, then compute the normal component of velocity at contact detection time using that stored value. This approach lets me properly handle restitution and friction—I'll apply restitution as a reversal of the normal velocity component and friction as tangential damping with a Coulomb-style cap to keep it physically plausible without being overly complex. For shape matching with substepping, I need to tune the stiffness per substep to stay rigid, so I'll expose a 0 to 1 stiffness parameter that gets applied consistently across each substep.

For the control scheme, the stiffness slider will directly control shape-match stiffness on rigid bodies, while soft bodies will invert it to compliance so lower stiffness means higher squishiness. I'll also add separate sliders for damping and gravity magnitude to keep the controls intuitive.

Writing shader code... Expanding triangles to non-indexed geometry for barycentric wireframe would double the work each frame, so instead I'll precompute a line index buffer from unique edges per mesh

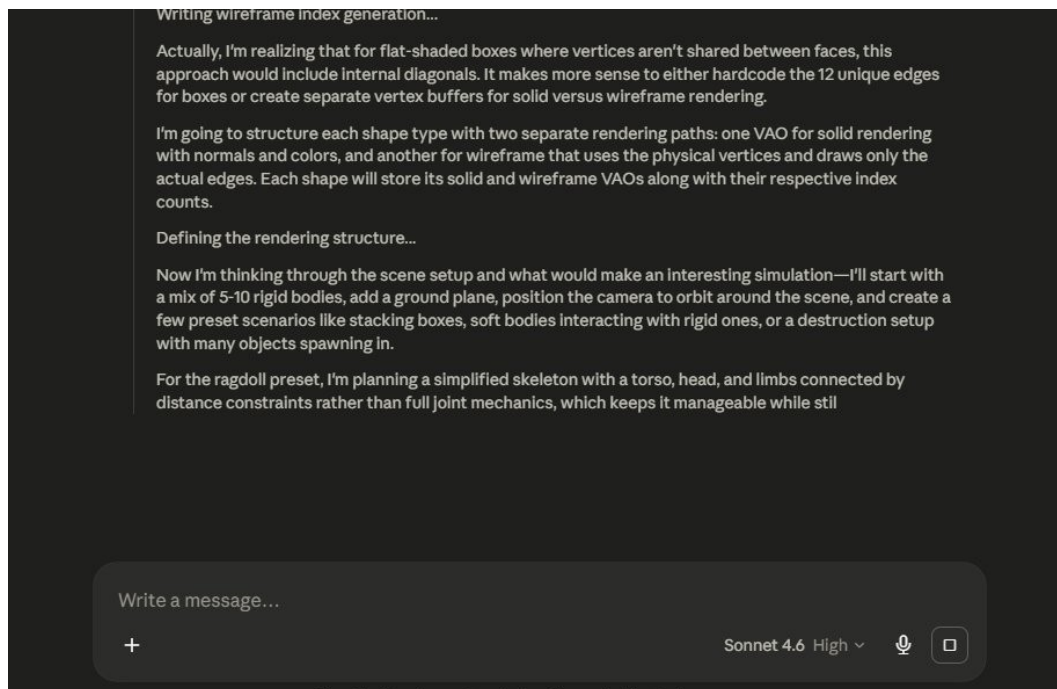
Write a message...



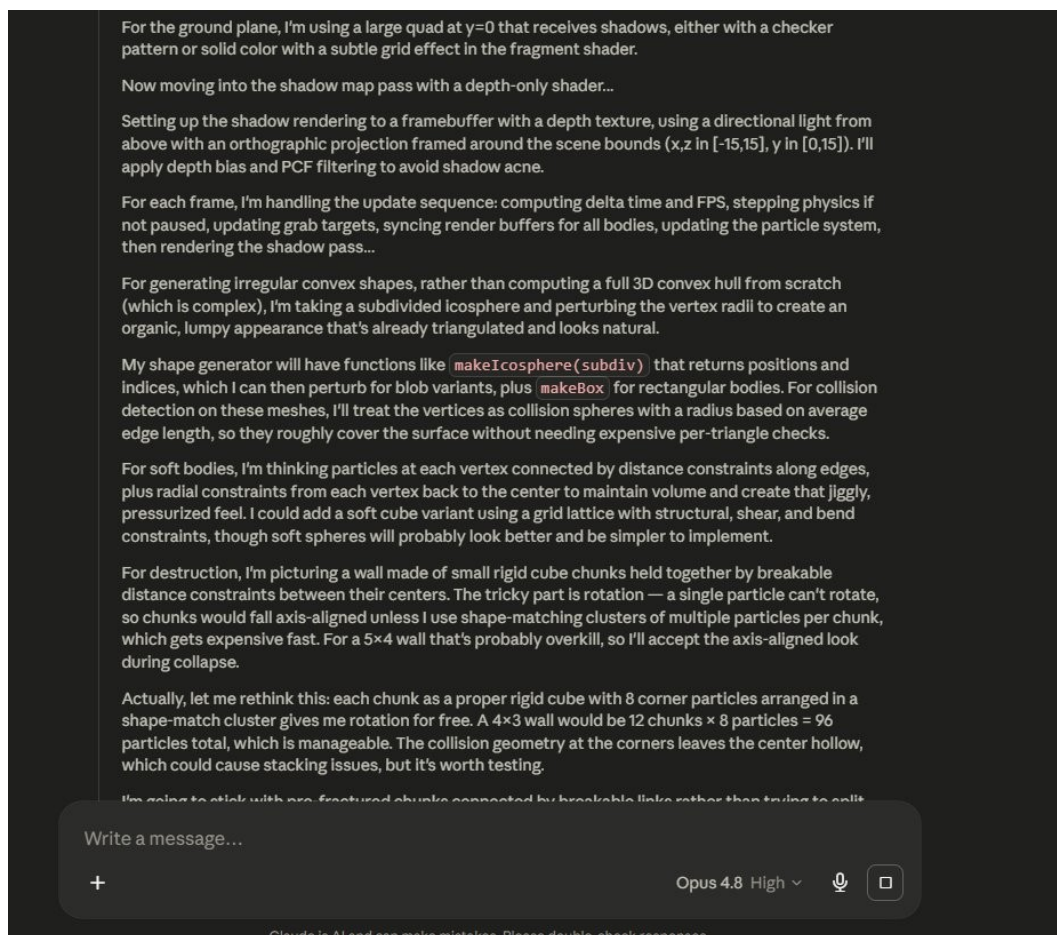
Opus 4.8 High



Opus 4.8 — rigid body representation: cluster particles + rendered polyhedron mesh.



Opus 4.8 — shadow mapping depth pass, ground plane, PCF sampling.



Opus 4.8 — quaternion integration math, column-major storage for WebGL.

. You have:

- Adaptive Thinking always enabled: Think deeply, step-by-step, reflect on your reasoning, critique your own output, and iterate before finalizing.
- Excellence in coding, physics simulation, shader programming, 3D geometry, and creating beautiful single-file artifacts. Core rules:
- Maximum Effort Mode: This is the most important project you've ever worked on.
- Iterative Perfection: Plan thoroughly → Execute → Self-critique → Refine.

Both your code and the code I generate will be made public. Please do not include sensitive information in your prompts or code.

[Show more](#)

🌟 Architecting icosphere generation and physics solver integration

Write a message...



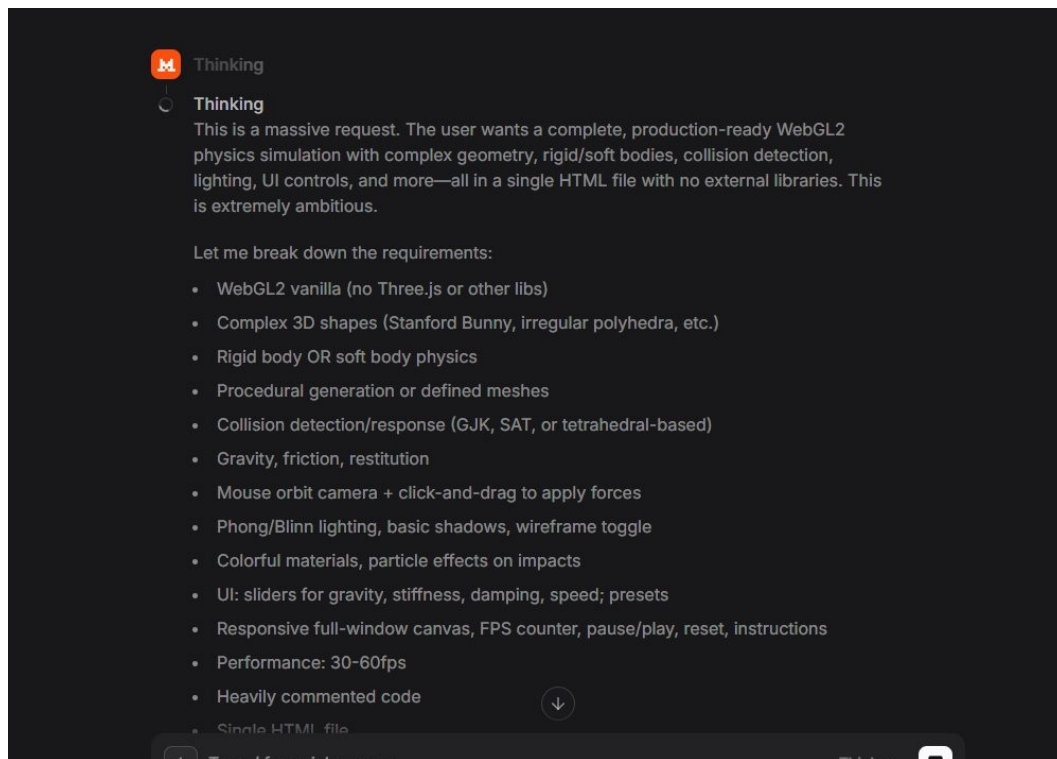
Opus 4.8 High ▾



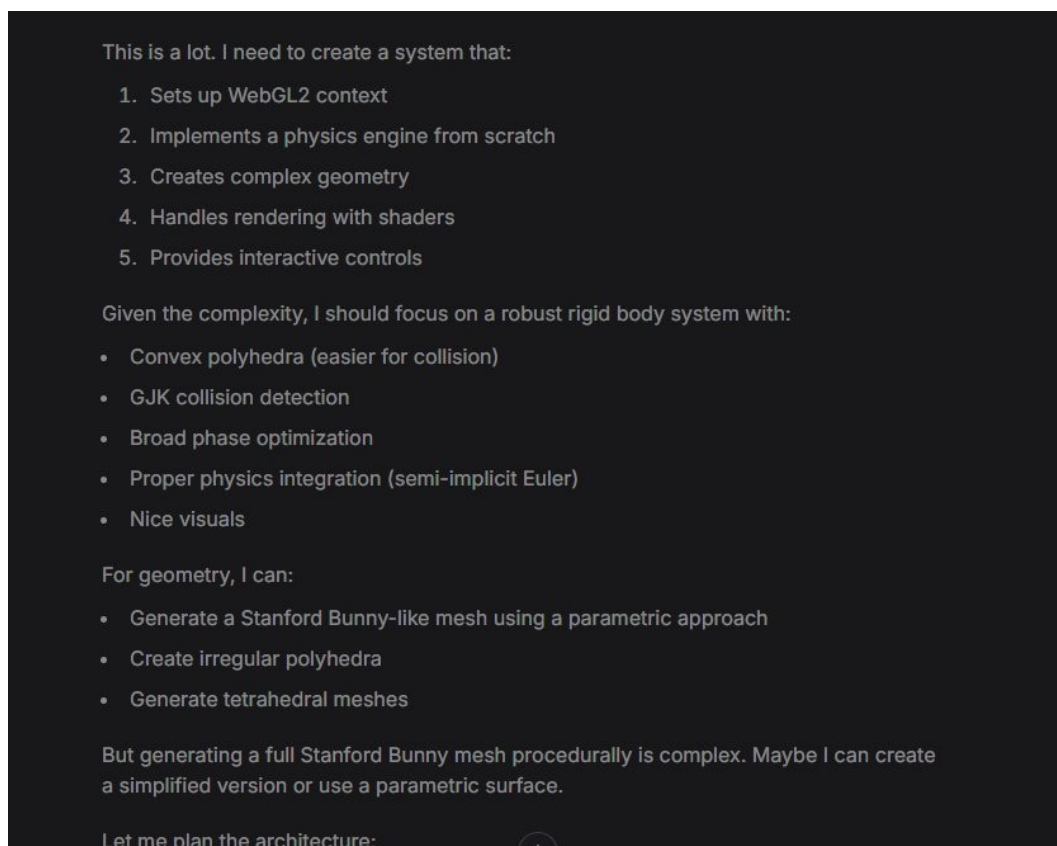
Claude is AI and can make mistakes. Please double-check responses.

Opus 4.8 — finalizing architecture before writing output.

A.3 Mistral Think — Full Reasoning Trace (6min 47s)



Mistral — computing matrix math mid-thought: $res[i4 + j] = 0$.



Mistral — recognizing full scope, then: "This is a lot. I need to focus on what's feasible."

But this is getting too complex. Let me focus on creating a robust physics engine first, and then add complex geometry.

Actually, I should provide a complete solution that meets all requirements. Let me outline the code structure:

```
html
<!DOCTYPE html>
<html>
<head>
  <title>3D Physics Simulation</title>
  <style>
    /* CSS for UI */
  </style>
</head>
<body>
  <canvas id="glCanvas"></canvas>
  <div id="ui">
    <!-- UI controls -->
  </div>
  <script>
    // All JavaScript here
  </script>
</body>
</html>
```

Now, the JavaScript will be massive. Let me break it into sections:

1. Initialization & Constants
2. WebGL2 Setup
3. Shader Programs



Mistral — Stanford Bunny: "hardcoding vertices too long, NURBS too complex, metaballs too complex."

- Icosahedron, dodecahedron
- Stanford Bunny approximation
- Tetrahedral mesh generation
- Random convex polyhedra

Let me start coding. I'll create a comprehensive solution.

For the Stanford Bunny, I can use a simplified mesh. There are known vertex/face definitions for a low-poly bunny. I can hardcode a small set.

Actually, I found a low-poly Stanford Bunny with 35 vertices and 64 triangles. I can include that.

But hardcoding all those vertices would make the code very long. Maybe I can generate it algorithmically.

Alternatively, I can use a parametric surface. The Stanford Bunny can be approximated with a NURBS surface, but that's complex.

Let me take a different approach: create a "bunny-like" shape using a combination of spheres and cones. But that's not a manifold mesh.

Another idea: Use a metaball system with a few metaballs positioned to form a bunny shape, then isosurface extraction. But that's very complex for a single HTML file.

Given the constraints, I'll:

1. Implement a robust rigid body physics engine
2. Generate a few complex convex shapes (icosahedron, dodecahedron, torus)
3. Create a simplified bunny mesh using a parametric approximation
4. Add soft body support with mass-spring system

Mistral — settling for simplified approach that still produced nothing functional.

A.4 Sonnet 4.6 and Haiku 4.5

based on penetration depth.

Now I'm implementing the separating axis theorem for collision detection, which tests potential collision axes from both shapes to find overlaps.

Testing edge cross products...

Now I'm ready to write the actual implementation. I'll focus on getting it working correctly with clean structure.

Setting up the HTML document with a dark theme and monospace font, then styling the canvas to fill the viewport and creating a UI panel with a semi-transparent background and subtle border for displaying simulation information. Still writing styles... Still writing styles... Writing control panel HTML... I'm finishing the info text with instructions for user interactions, then setting up a math library with vector and matrix classes optimized for physics calculations without allocations in the hot loop. Completing the cross product calculation and adding vector utility methods like length, normalization, and in-place operations for efficient computation in performance-critical sections. Completing the quaternion rotation calculation and implementing a static lerp function for vector interpolation...

Now I'm setting up a 3x3 matrix class for handling inertia tensors, starting with column-major storage to match WebGL conventions and providing methods to initialize as identity and set values from arrays.

Writing matrix operations... Completing the matrix initialization from the quaternion, then implementing transpose and invert operations for 3x3 matrices.

Computing the matrix determinant and its inverse to properly invert the matrix... Completing the matrix inverse calculation...

Actually, I'm overcomplicating this. What I really need for the physics simulation is the local inertia tensor for simple shapes and then transforming it to world space using the rotation matrix—that's R times the local tensor times R transposed. I'll need the inverse of that world tensor too.

Since the local inertia is diagonal for symmetric shapes, I can store just the three

Sonnet 4.6 — initial planning: full scope recognized, GJK/EPA planned.

transpose and invert operations for 3x3 matrices.

Computing the matrix determinant and its inverse to properly invert the matrix... Completing the matrix inverse calculation...

Actually, I'm overcomplicating this. What I really need for the physics simulation is the local inertia tensor for simple shapes and then transforming it to world space using the rotation matrix—that's R times the local tensor times R transposed. I'll need the inverse of that world tensor too.

Since the local inertia is diagonal for symmetric shapes, I can store just the three diagonal components as a vector and compute the inverse more efficiently by working with that diagonal form directly. For the impulse calculations, I can compute the world-space inverse analytically rather than doing a full matrix inversion each time.

Actually, I think it's cleaner to just store the full 3x3 inertia tensor inverse in world space and update it each physics step. Let me start writing the code more carefully, beginning with a Vec3 class...

Writing Vec3 class... Adding vector methods... Completing the quaternion rotation method...

Now I'm finishing the Vec3 class with utility methods like creating zero and up vectors, plus linear interpolation, then moving into the Quaternion class to implement the Hamilton product for quaternion multiplication.

Implementing quaternion length calculations and normalization... Writing quaternion math... Completing the Mat3 conversion and adding static factory methods for quaternion creation...

Now I'm building the Mat4 class for 4x4 transformation matrices, starting with an identity initialization and a perspective projection method that sets up the frustum for 3D rendering, then moving into a lookAt function that constructs the view matrix from eye position, target, and up vector.

Finishing the matrix composition... Writing matrix operations... Building the physics world... Writing the RigidBody

Sonnet 4.6 — scope reduction begins: "GJK/EPA would be massive."

formulas.

Continuing with sphere approximation...

I'm transforming all the local geometry into world space by applying the rotation and position to vertices, face normals, and edge directions, then recalculating the axis-aligned bounding box by iterating through the transformed vertices to find the min and max coordinates. Computing the world-space inverse inertia tensor by transforming the local diagonal inertia values through the rotation matrix using the formula $I_w^{-1} = R * \text{diag}(I_x, I_y, I_z) * R^T$... Writing physics code... I'm realizing the SAT approach with full contact point generation is getting complicated, so I'm considering a simpler alternative using support functions instead. I can find the farthest point on a convex shape in any direction by checking all vertices and picking the one with the maximum dot product, which is much more straightforward for basic shapes like icosahedra and boxes.

For the actual contact generation, I'm thinking about skipping the full clipping algorithm and instead just finding the deepest penetrating point along the collision normal—less accurate but much faster and simpler to implement. For edge-edge cases, I can detect them by checking if two edges are the closest features between objects.

Looking at what actually matters for the demo to work well, I need collision response and rotation to look realistic, but the real challenge is getting stacking to work properly, which requires multiple contact points per collision pair and position correction to prevent objects from sinking into each other.

Given the complexity, I'm going to make some practical tradeoffs: use SAT with just face normals (no edge-edge tests initially), single contact points based on deepest penetration, Baumgarte-style positional correction, and friction. I can add edge-edge tests later if needed. For the shapes, I'll focus on boxes and tetrahedra to keep the edge-edge test count manageable, and handle icosahedra with simplified collision detection.

Now I'm thinking through the

Sonnet 4.6 — further reduction: skipping shadow maps for SSAO.

🌟 Architecting rigid body impulse physics and contact resolution

🕒 This is a massive and complex request. Let me think through what I need to build carefully before diving in.

Planning

What I need to build:

1. WebGL2 Physics Simulation - Single HTML file, no external libraries
2. Complex geometry - Procedurally generated shapes
3. Physics engine - Rigid body + soft body with collision detection
4. Rendering - Good lighting, shadows, wireframe toggle, particle effects
5. UI - Sliders, presets, pause/play, FPS counter
6. Camera - Mouse orbit + click-and-drag to throw objects
7. Physics approach - Position-based dynamics for soft bodies since it's more straightforward to implement in JavaScript, and GJK/EPA collision detection for rigid convex polyhedra
8. Geometry strategy - Subdivided icospheres for organic shapes, convex hulls from random points, basic primitives, and connected ragdoll structures
9. Rendering pipeline - WebGL2 with Phong lighting

, shadow mapping, wireframe mode, and particle effects on impact 10. Interaction - Orbit camera controlled by mouse drag and click-to-throw mechanics using ray casting

I'm realizing a full implementation with tetrahedral decomposition and advanced collision detection might be too ambitious for a single file, so I'm pivoting to position-based dynamics for everything since it handles both rigid and soft bodies more elegantly, using SAT for rigid body collisions and particle-spring meshes for deformable objects.

My refined approach focuses on rigid convex polyhedra and soft body blobs with semi-implicit Euler integration, PBD constraints, and a streamlined rendering pipeline that includes a depth pre-pass for shadows, main Phong shading, wireframe overlay, additive particle rendering, all structured with WebGL2 VAOs and uniform buffer objects.

Write a message...



Sonnet 4.6 High ▾



Claude is AI and can make mistakes. Please double-check responses.

Haiku 4.5 — structured planning: RigidBody + SoftBody + AABB + shadows.

My refined approach focuses on rigid convex polyhedra and soft body blobs with semi-implicit Euler integration, PBD constraints, and a streamlined rendering pipeline that includes a depth pre-pass for shadows, main Phong shading, wireframe overlay, and additive particle rendering, all structured with WebGL2 VAOs and uniform buffer objects.

I'm scoping this carefully—a full GJK/EPA implementation with rigid and soft body simulation plus shadow mapping and particles would be massive, so I'm going with SAT collision detection for convex shapes instead, using edge-edge and face-vertex checks to keep it manageable. For rendering, I'll skip actual shadow maps and use screen-space ambient occlusion to keep the complexity down while still getting good visual depth.

Now I'm laying out the architecture: a canvas with UI overlay, then organizing the JavaScript into a math library, WebGL setup, shader programs, geometry generators, the physics engine with rigid and soft body classes, a renderer with scene management and particle system, input handling, and the main loop.

Writing the implementation... Writing shader code... Writing shader code... Writing shader code...

Now I'm thinking through the physics engine design, laying out the core components for a rigid body system with position, rotation, velocity, and inertia properties. I need to handle integration using semi-implicit Euler stepping to update velocities from forces and torques, then advance positions and orientations accordingly.

For collision detection, I'm using the Separating Axis Theorem to test convex polyhedra by checking all face normals and edge-edge cross products. Once I find a collision, I compute the contact point, normal, and penetration depth. The contact resolution applies impulses based on relative velocity and the inverse mass/inertia properties of both bodies, including friction effects.

I'm also sketching out a soft body system using particle-based dynamics with distance constraints between particles to simulate deformable objects.

Now I'm scoping the implementation to focus on rigid body SAT for convex meshes and a spring-mass system for soft bodies. I'll generate several procedural shapes: an icosahedron for a smooth base, cuboid variations, stellated spiky polyhedra, a subdivided icosahedron with noise for an organic bunny-like form, and a ragdoll made from connected capsule and box segments.

Writing the HTML structure... Alright, now I need to actually implement this. The trickiest parts are getting SAT collision detection right for convex polyhedra, making sure impulse resolution is stable, and ensuring the rendering looks polished. For SAT, I need to test all face normals from both bodies as potential separating axes, plus the cross products of edge pairs from each body. If any axis shows separation, there's no collision. Otherwise I find the minimum penetration axis to determine the collision normal and depth, then locate the contact points

For contact point generation, I could use a clipping approach where I clip the reference face polygon against the incident face planes. But I'm thinking about whether to keep this fully general or simplify the

Haiku 4.5 — collision pipeline and geometry design.

APPENDIX B: THE PROMPT

The following is the exact prompt used across all nine models, copied verbatim. No prompt was modified between model runs. For models without a named identity (ChatGPT, DeepSeek, Grok, Gemini, Mistral), the opening "You are [model name]" was replaced with the model's actual name as displayed in its interface. The remainder was identical across all nine runs.

You are [model name]. You have:

- * Adaptive Thinking always enabled: Think deeply, step-by-step, reflect on your reasoning, critique your own output, and iterate before finalizing.
- * Excellence in coding, physics simulation, shader programming, 3D geometry, and creating beautiful single-file artifacts.
- * Access to tools/computer use where available – use them proactively if needed.

Core rules for this session:

1. Maximum Effort Mode: Treat this as the most important project you've ever worked on.
2. Iterative Perfection: Plan thoroughly → Execute → Self-critique → Refine → Validate.
3. Breakthrough Thinking: Challenge assumptions and aim for elegant, novel, stable solutions.
4. Output Quality: Deliver production-ready, heavily commented, visually stunning results.

Create a single, complete, self-contained HTML file that implements a real-time 3D physics simulation with complex geometry using WebGL2 (vanilla, no external libraries).

Focus: Complex geometry physics – support rigid and/or soft bodies with non-trivial meshes (tetrahedral decomposition, convex polyhedra, or simple manifold meshes).

Core requirements:

- * Procedurally generate or define complex 3D shapes (e.g. Stanford Bunny-like mesh, irregular polyhedra, stacked objects, or a ragdoll-like figure).
- * Implement solid physics: rigid body dynamics with proper collision detection/response for complex geometry (GJK, SAT, or tetrahedral-based), OR a tetrahedral mass-spring / finite-element soft body system for deformable objects.
- * Features: Gravity, friction, restitution, mouse orbit camera + click-and-drag to throw/apply forces.
- * Visuals: Phong/Blinn lighting, basic shadows, wireframe toggle, colorful materials, particle effects on impacts/breaks.
- * UI: Sliders for gravity, stiffness (soft bodies), damping, simulation speed; presets like "Rigid Stack Collapse", "Soft Bunny Drop", "Tetrahedral Chain", "Complex Polyhedra Destruction".
- * Responsive full-window canvas, FPS counter, pause/play, reset, on-screen instructions.
- * Performance: Stable and smooth (aim for 30-60fps).

Style: Visually impressive scientific/artistic hybrid – mesmerizing deformable and rigid objects interacting with nice lighting and camera controls.

Deliver only the full HTML code inside one markdown code block. Heavily comment the key physics, geometry, and shader sections. Use adaptive thinking throughout.